SQL JOIN LAB

Table Creation

```
CREATE TABLE CUSTOMER1_24BYB1117 (
CUST_ID NUMBER PRIMARY KEY,
CUST_NAME VARCHAR2(50),
CITY VARCHAR2(50)
);

CREATE TABLE ORDERS_24BYB1117 (
ORDER_ID NUMBER PRIMARY KEY,
CUST_ID NUMBER,
ORDER_DATE DATE,
AMOUNT NUMBER
);

CREATE TABLE PRODUCT_24BYB1117 (
PRODUCT_ID NUMBER PRIMARY KEY,
PRODUCT_NAME VARCHAR2(50),
CATEGORY VARCHAR2(50)
);

CREATE TABLE ORDERITEM_24BYB1117 (
ORDER_ID NUMBER,
PRODUCT_ID NUMBER,
QUANTITY NUMBER
);
```

Insert Multiple Records

```
INSERT ALL
INTO CUSTOMER1_24BYB1117 VALUES (1,'Arun','Chennai')
INTO CUSTOMER1_24BYB1117 VALUES (2,'Divya','Bangalore')
INTO CUSTOMER1_24BYB1117 VALUES (3,'Karthik','Hyderabad')
INTO CUSTOMER1_24BYB1117 VALUES (4,'Meena','Delhi')
INTO CUSTOMER1_24BYB1117 VALUES (5,'Rahul','Mumbai')
SELECT * FROM DUAL;
INSERT ALL
INTO ORDERS_24BYB1117 VALUES (101,1,DATE '2024-01-10',5000)
INTO ORDERS_24BYB1117 VALUES (102,2,DATE '2024-01-11',7000)
INTO ORDERS_24BYB1117 VALUES (103,1,DATE '2024-01-15',2000)
INTO ORDERS_24BYB1117 VALUES (104,3,DATE '2024-01-20',9000)
INTO ORDERS_24BYB1117 VALUES (105,5,DATE '2024-01-25',3000)
SELECT * FROM DUAL;
INSERT ALL
INTO PRODUCT_24BYB1117 VALUES (201,'Laptop','Electronics')
INTO PRODUCT_24BYB1117 VALUES (202,'Mouse','Electronics')
INTO PRODUCT_24BYB1117 VALUES (203,'Chair','Furniture')
INTO PRODUCT_24BYB1117 VALUES (204,'Table','Furniture')
```

```
INTO PRODUCT_24BYB1117 VALUES (205,'Headphones','Electronics')
SELECT * FROM DUAL;
INSERT ALL
INTO ORDERITEM_24BYB1117 VALUES (101,201,1)
INTO ORDERITEM_24BYB1117 VALUES (101,202,2)
INTO ORDERITEM_24BYB1117 VALUES (102,203,1)
INTO ORDERITEM_24BYB1117 VALUES (103,205,1)
INTO ORDERITEM_24BYB1117 VALUES (104,204,1)
SELECT * FROM DUAL;
```

PRACTICE QUESTIONS

B. NATURAL JOIN

7. Display all customer names and their order amounts using **NATURAL JOIN**.

```
SELECT CUST_NAME, AMOUNT
FROM CUSTOMER1_24BYB1117
NATURAL JOIN ORDERS_24BYB1117;
```

8. Show product names and quantities sold.

```
SELECT PRODUCT_NAME, QUANTITY
FROM PRODUCT_24BYB1117
NATURAL JOIN ORDERITEM_24BYB1117;
```

9. List order IDs with product categories.

```
SELECT ORDER_ID, CATEGORY
FROM PRODUCT_24BYB1117
NATURAL JOIN ORDERITEM_24BYB1117;
```

10. Display customers who placed more than one order.

```
SELECT CUST_NAME
FROM CUSTOMER1_24BYB1117
NATURAL JOIN ORDERS_24BYB1117
GROUP BY CUST_NAME
HAVING COUNT(*) > 1;
```

11. Show orders with electronics items.

```
SELECT ORDER_ID, PRODUCT_NAME
FROM PRODUCT_24BYB1117
NATURAL JOIN ORDERITEM_24BYB1117
WHERE CATEGORY='Electronics';
```

12. Display all furniture orders.

```
SELECT ORDER_ID, PRODUCT_NAME
FROM PRODUCT_24BYB1117
NATURAL JOIN ORDERITEM_24BYB1117
WHERE CATEGORY='Furniture';
```

C. LEFT OUTER JOIN

13. List all customers and their order amounts.

```
SELECT C.CUST_NAME, O.AMOUNT
FROM CUSTOMER1_24BYB1117 C
LEFT JOIN ORDERS_24BYB1117 O
ON C.CUST_ID = O.CUST_ID;
```

14. Show customers who never placed an order.

```
SELECT C.CUST_NAME
FROM CUSTOMER1_24BYB1117 C
LEFT JOIN ORDERS_24BYB1117 O
ON C.CUST_ID = O.CUST_ID
WHERE O.ORDER_ID IS NULL;
```

15. Display all orders and product names.

```
SELECT O.ORDER_ID, P.PRODUCT_NAME
FROM ORDERS_24BYB1117 O
LEFT JOIN ORDERITEM_24BYB1117 OI
ON O.ORDER_ID = OI.ORDER_ID
LEFT JOIN PRODUCT_24BYB1117 P
ON OI.PRODUCT_ID = P.PRODUCT_ID;
```

16. Identify orders without items.

```
SELECT O.ORDER_ID
FROM ORDERS_24BYB1117 O
LEFT JOIN ORDERITEM_24BYB1117 OI
ON O.ORDER_ID = OI.ORDER_ID
WHERE OI.PRODUCT_ID IS NULL;
```

17. Display all products and order IDs.

```
SELECT P.PRODUCT_NAME, OI.ORDER_ID
FROM PRODUCT_24BYB1117 P
LEFT JOIN ORDERITEM_24BYB1117 OI
ON P.PRODUCT_ID = OI.PRODUCT_ID;
```

18. Show customers with NULL order values.

```
SELECT C.CUST_NAME
FROM CUSTOMER1_24BYB1117 C
```

```
LEFT JOIN ORDERS_24BYB1117 O
ON C.CUST_ID = O.CUST_ID
WHERE O.AMOUNT IS NULL;
```

D. RIGHT OUTER JOIN

19. List all orders with customer names.

```
SELECT C.CUST_NAME, O.ORDER_ID
FROM CUSTOMER1_24BYB1117 C
RIGHT JOIN ORDERS_24BYB1117 O
ON C.CUST_ID = O.CUST_ID;
```

20. Identify orders with missing customers.

```
SELECT O.ORDER_ID
FROM CUSTOMER1_24BYB1117 C
RIGHT JOIN ORDERS_24BYB1117 O
ON C.CUST_ID = O.CUST_ID
WHERE C.CUST_ID IS NULL;
```

21. Display products that were never ordered.

```
SELECT P.PRODUCT_NAME
FROM PRODUCT_24BYB1117 P
RIGHT JOIN ORDERITEM_24BYB1117 OI
ON P.PRODUCT_ID = OI.PRODUCT_ID
WHERE OI.ORDER_ID IS NULL;
```

22. Show all furniture items and order IDs.

```
SELECT P.PRODUCT_NAME, OI.ORDER_ID
FROM PRODUCT_24BYB1117 P
RIGHT JOIN ORDERITEM_24BYB1117 OI
ON P.PRODUCT_ID = OI.PRODUCT_ID
WHERE CATEGORY='Furniture';
```

23. Display products with NULL orders.

```
SELECT P.PRODUCT_NAME
FROM PRODUCT_24BYB1117 P
LEFT JOIN ORDERITEM_24BYB1117 OI
ON P.PRODUCT_ID = OI.PRODUCT_ID
WHERE OI.ORDER_ID IS NULL;
```

24. Show electronics products without orders.

```
SELECT P.PRODUCT_NAME
FROM PRODUCT_24BYB1117 P
LEFT JOIN ORDERITEM_24BYB1117 OI
```

```
ON P.PRODUCT_ID = OI.PRODUCT_ID
WHERE CATEGORY='Electronics'
AND OI.ORDER_ID IS NULL;
```

E. FULL OUTER JOIN

25. Show all customers and all orders.

```
SELECT C.CUST_NAME, O.ORDER_ID
FROM CUSTOMER1_24BYB1117 C
FULL OUTER JOIN ORDERS_24BYB1117 O
ON C.CUST_ID = O.CUST_ID;
```

26. Identify customers without orders and orders without customers.

```
SELECT C.CUST_NAME, O.ORDER_ID
FROM CUSTOMER1_24BYB1117 C
FULL OUTER JOIN ORDERS_24BYB1117 O
ON C.CUST_ID = O.CUST_ID
WHERE C.CUST_ID IS NULL
OR O.CUST_ID IS NULL;
```

27. Display all products and order IDs.

```
SELECT P.PRODUCT_NAME, OI.ORDER_ID
FROM PRODUCT_24BYB1117 P
FULL OUTER JOIN ORDERITEM_24BYB1117 OI
ON P.PRODUCT_ID = OI.PRODUCT_ID;
```

28. Identify unordered products.

```
SELECT P.PRODUCT_NAME
FROM PRODUCT_24BYB1117 P
FULL OUTER JOIN ORDERITEM_24BYB1117 OI
ON P.PRODUCT_ID = OI.PRODUCT_ID
WHERE OI.ORDER_ID IS NULL;
```

29. Show complete customer–order relationship.

```
SELECT *
FROM CUSTOMER1_24BYB1117 C
FULL OUTER JOIN ORDERS_24BYB1117 O
ON C.CUST_ID = O.CUST_ID;
```

30. Display unmatched records from both tables.

```
SELECT *
FROM CUSTOMER1_24BYB1117 C
FULL OUTER JOIN ORDERS_24BYB1117 O
ON C.CUST_ID = O.CUST_ID
```

```
WHERE C.CUST_ID IS NULL
OR O.CUST_ID IS NULL;
```

F. CROSS JOIN

31. Generate all customer–order combinations.

```
SELECT *
FROM CUSTOMER1_24BYB1117
CROSS JOIN ORDERS_24BYB1117;
```

32. Generate all customer–product combinations.

```
SELECT *
FROM CUSTOMER1_24BYB1117
CROSS JOIN PRODUCT_24BYB1117;
```

33. Generate all product–order combinations.

```
SELECT *
FROM PRODUCT_24BYB1117
CROSS JOIN ORDERS_24BYB1117;
```

34. Count total rows produced by cross join.

```
SELECT COUNT(*)
FROM CUSTOMER1_24BYB1117
CROSS JOIN PRODUCT_24BYB1117;
```

35. Use WHERE clause to limit cross join output.

```
SELECT *
FROM CUSTOMER1_24BYB1117 C
CROSS JOIN ORDERS_24BYB1117 O
WHERE C.CUST_ID = O.CUST_ID;
```

36. Explain why cross join is risky for large tables.

Cross join produces the **Cartesian product** of two tables.
If table A has *m* rows and table B has *n* rows, the result will contain **m × n rows**, which can become very large and slow for big datasets.